Colegio Tecnológico UTN · Diseño Backend · 5.º año · Comisión Berdichevsky

Práctica — El front: los productos en la página

Unidad 7 · Clase 3 · Conectar el catálogo con fetch: tarjetas, filtro y carrito

En esta práctica tu tienda deja de ser una API que solo responde en Thunder Client: conectás el front y los productos aparecen en la página. El backend ya quedó completo en la Clase 2, así que hoy todo el trabajo pasa por un solo archivo, **public/app.js**. Al terminar vas a tener el catálogo con imágenes, el filtro por categoría y el carrito funcionando.

Trabajamos sobre una librería como ejemplo. Si tu tienda es de otro rubro, los pasos son los mismos: cambian los productos y las categorías.

Paso 0 — Punto de partida

La tienda tiene tres piezas y cada una habla solo con la de al lado. El navegador le pide datos a tu API, y la API se los pide a Supabase:

```
Navegador (public/app.js)    // HOY: pide los datos con fetch
      |  GET /productos
Tu API en Express            // Clase 2: rutas, controllers y models
      |  .from('productos').select(...)
Supabase                     // Clase 1: tablas, Auth y Storage
```

Las tres piezas viajan juntas: el mismo index.js que levanta la API también entrega la página, con esta línea que ya está en la plantilla:

```
// entrega index.html, estilos.css y app.js
app.use(express.static(path.join(__dirname, 'public')));
```

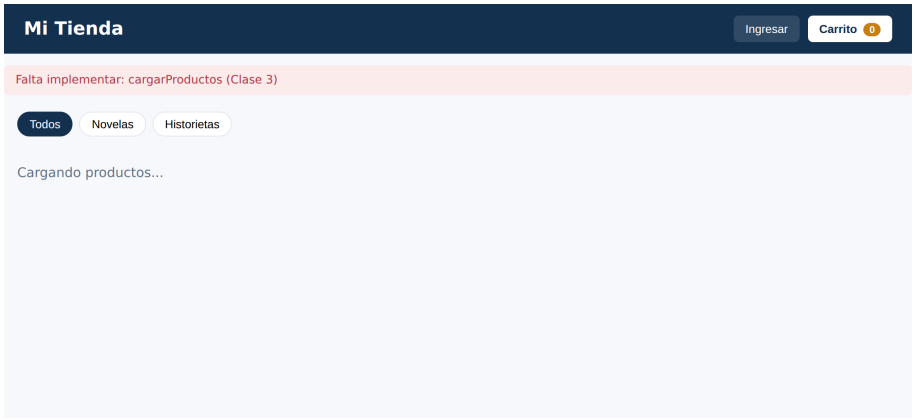
> ⚠ **¿Por qué el front no le habla directo a Supabase?**
>
> Porque para entrar a la base hace falta la **clave secreta**, y todo lo que está en app.js lo puede leer cualquiera con F12. La clave vive en el .env del servidor y nunca sale de ahí. El front no sabe que Supabase existe: solo conoce las rutas de tu API.

Paso 1 — Abrir la tienda como se debe

Levantá el servidor desde la carpeta del proyecto:

```
npm start
// Tienda en http://localhost:3000
```

Entrá a **http://localhost:3000**. No abras index.html con doble clic: así el archivo se abre sin servidor y ningún fetch encuentra la API. Tendrías que ver esto:



- arriba, los botones de categorías: **Todos** y las de tu rubro;
- en el centro, “Cargando productos…”;
- un aviso rojo: **Falta implementar: cargarProductos (Clase 3)**.

Las categorías ya aparecen porque **cargarCategorias** viene resuelta en la plantilla. Mirala antes de seguir: hace lo mismo que vas a escribir vos, con otra ruta.

Paso 2 — fetch en tres pasos

fetch es Thunder Client adentro del navegador: manda un pedido a una ruta y espera la respuesta. Lo que hacías a mano con un clic en Send, ahora lo hace app.js solo.

```
const res = await fetch('/productos');   // 1. pedir: GET a la API
if (!res.ok) { ... }                     // 2. revisar: ¿salió bien?
const productos = await res.json();      // 3. leer: el JSON pasa a ser un array
```

¿Qué hace cada parte?

Código	Qué hace
`await fetch('/productos')`	manda un GET a tu API. Si no decís otra cosa, fetch siempre manda GET
`res.ok`	vale true si el código está entre 200 y 299; con 400, 404, 500 o 501 vale false
`await res.json()`	convierte el texto JSON en un array de objetos de JavaScript

Una comparación para recordarlo: pedir en un delivery. **fetch** es hacer el pedido, **res** es la caja que llega a tu puerta y **res.json()** es abrirla. Antes de abrirla conviene mirar la etiqueta (**res.ok**): si dice “pedido rechazado”, adentro no hay comida sino un mensaje.

> ⚠ **Los dos await son obligatorios.**
> El primero espera que llegue la respuesta. El segundo espera que termine de leerse el contenido. Si te olvidás de alguno, en vez de los datos tenés una promesa (Promise) y nada se dibuja.

¿Por qué '/productos' y no 'http://localhost:3000/productos'? Una ruta que empieza con / se pide al mismo servidor que entregó la página. Así el mismo código anda en tu compu y, cuando la subamos, en Vercel.

Paso 3 — Completar cargarProductos

En public/app.js, buscá la función. Así viene:

```
async function cargarProductos(categoriaId) {
  // TODO (Clase 3): completar esta función.
  mostrarMensaje('Falta implementar: cargarProductos (Clase 3)', 'error');
}
```

Borrá las dos líneas del TODO y escribí esto:

```
async function cargarProductos(categoriaId) {
  categoriaActual = categoriaId || null;   // para marcar el botón activo
  let url = '/productos';
  if (categoriaId) url += '?categoria=' + categoriaId;

  const res = await fetch(url);
  if (!res.ok) return mostrarMensaje(await res.text(), 'error');

  const productos = await res.json();
  pintarProductos(productos);       // ya resuelta
  pintarCategoriaActiva();          // ya resuelta
}
```

¿Qué hace cada parte?

Código	Qué hace
`categoriaActual = categoriaId || null`	recuerda qué categoría se eligió; si no vino ninguna, queda en null ("Todos")
`let url`	la dirección se arma de a partes, igual que la consulta del model en la Clase 2
`'?categoria=' + categoriaId`	agrega el filtro a la dirección; del otro lado llega como req.query.categoria
`if (!res.ok) return ...`	si la API respondió con error, muestra su mensaje y corta: no intenta dibujar nada
`await res.text()`	lee el mensaje de error, que llega como texto y no como JSON
`pintarProductos(productos)`	arma una tarjeta por producto con innerHTML

✓ Guardá y recargá con Ctrl + F5.

No hace falta reiniciar el servidor: solo cambiaste el front. Los productos tienen que aparecer con su foto, su categoría, su precio y su stock.

Paso 4 — Probar el filtro y seguir un clic

Tocá una categoría: tienen que quedar solo sus productos y el botón tiene que quedar marcado. Tocá **Todos** y vuelven todos. Esto es lo que pasa con un solo clic en "Novelas":

1. El botón tiene onclick="cargarProductos(2)": lo armó cargarCategorias con el id de cada categoría.
2. cargarProductos arma la dirección /productos?categoria=2 y hace el fetch.
3. La ruta GET /productos llama a verProductos, que lee req.query.categoria y lo pasa a número.

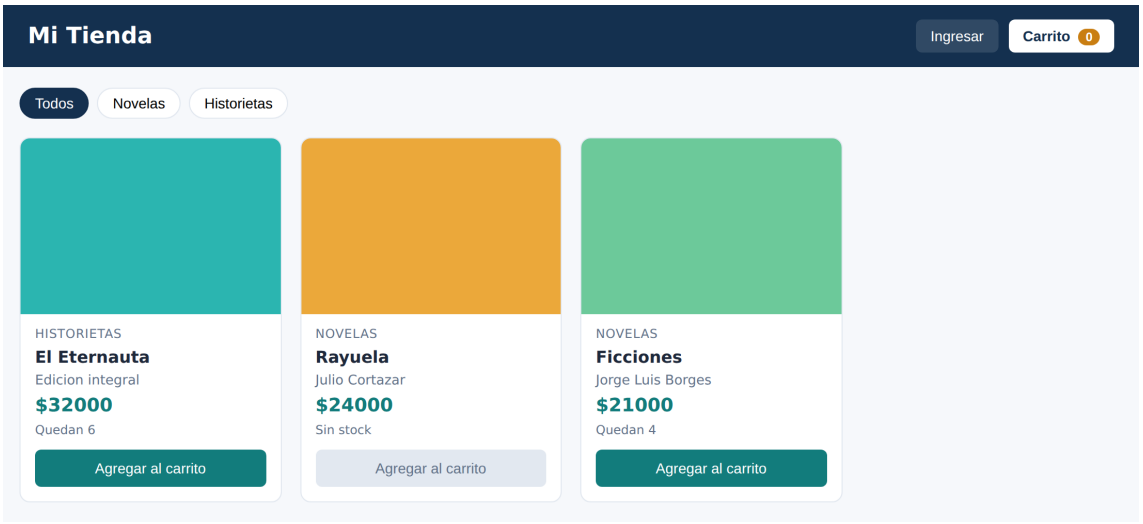
4. obtenerProductos(2) le agrega a la consulta .eq('categoria_id', 2) y Supabase devuelve las filas.
5. El controller responde 200 con el JSON y el front lo convierte en tarjetas.

Los pasos 3 y 4 son el código del backend que escribiste la clase pasada. Hoy solo agregaste la punta que faltaba.

Del JSON a la tarjeta. pintarProductos usa un template literal: lo que va entre ${ } se reemplaza por el dato de cada producto.

Dato del JSON	Dónde aparece en la tarjeta
`p.imagen_url`	el src de la imagen (la URL pública de Storage)
`p.categorias.nombre`	la etiqueta de arriba, gracias al categorias(nombre) del select
`p.nombre`, `p.descripcion`	el título y el texto
`p.precio`	el precio, con el $ adelante
`p.stock`	"Quedan 6", o "Sin stock" y el botón desactivado si es 0
`p.id`	el onclick="agregar(1)" del botón: así el carrito sabe qué producto es

Así tiene que quedar:



Paso 5 — Mirar adentro con F12

Apretá **F12** y abrí la pestaña **Red** (Network). Filtrá por **Fetch/XHR** y recargá la página.

Pestaña	Qué vas a ver
Red	dos pedidos, categorias y productos, los dos con estado 200. Si tocás una categoría, aparece productos?categoria=2
Red → Respuesta	el mismo JSON que veías en Thunder Client
Consola	los errores de app.js en rojo, con el número de línea donde se cortó

✓ **F12 es tu Thunder Client en el navegador.**

Ahí ves qué pidió el front y qué le respondió la API. Cuando algo no se dibuja, F12 es lo primero que se abre.

Paso 6 — El carrito ya anda

Tocá **Agregar al carrito** en dos productos y abrí el carrito. Se puede sumar y restar con los botones + y −. Recargá la página: el carrito sigue ahí.

El carrito no toca la base: vive en el navegador, en **localStorage**. En F12, pestaña **Aplicación** (Application) → Local Storage, vas a ver la clave carrito:

```
[{"producto_id":1,"cantidad":2},{"producto_id":3,"cantidad":1}]
```

⚠ **Guarda solo ids y cantidades, nunca precios.**

El "Total estimado" es para mostrar. El total de verdad lo calcula crearPedido en el backend, con los precios de la base, en el momento de comprar. El botón Comprar todavía dice "Falta implementar": es lo de la próxima clase.

Paso 7 — Hacela tuya

Con el catálogo andando, adaptá la tienda a tu rubro. Son tres lugares:

Qué	Dónde
Nombre de la tienda	el <title> y el <h1> de public/index.html
Colores	las variables del principio de public/estilos.css: --principal, --acento y --destacado
Categorías, productos e imágenes	el Table Editor de Supabase y el bucket productos de Storage

✓ **Mínimo para hoy: 6 productos en 2 categorías, todos con imagen.**

Si cambiás algo en Supabase no hace falta tocar código: recargás la página y aparece.

Errores frecuentes

No carga nada, ni las categorías

✕ **Causa:** abriste index.html con doble clic y la dirección empieza con file://.

✓ **Solución:** entrá por http://localhost:3000 con el servidor levantado.

Sigue el aviso “Falta implementar: cargarProductos”

✕ **Causa:** no guardaste, quedó la línea del TODO o el navegador usa la versión vieja.

✓ **Solución:** guardá, borrá el mostrarMensaje del TODO y recargá con Ctrl + F5.

Aparece “Error al obtener los productos”

✕ **Causa:** la API respondió 500: falló la consulta a Supabase.

✓ **Solución:** mirá la terminal del servidor; revisá el .env y obtenerProductos de la Clase 2.

Las tarjetas salen con la imagen rota

✕ **Causa:** imagen_url está vacío, mal copiado o el bucket no es público.

✓ **Solución:** abrí la URL en otra pestaña: si no muestra la foto, corregila o hacé público el bucket.

En Consola: productos.forEach is not a function

✕ **Causa:** falta el await antes de res.json().

✓ **Solución:** const productos = await res.json();

El filtro no cambia nada

✕ **Causa:** el model no aplica .eq('categoria_id', categoriaId).

✓ **Solución:** revisá el if (categoriaId) de obtenerProductos.

Extensión opcional — Buscador por nombre

El buscador es uno de los extras del TP. Busca dentro de los productos que ya llegaron, así que no necesita una ruta nueva ni hace ningún fetch.

1. En index.html, arriba de la grilla de productos:

```
<input id="buscador" class="buscador" type="search"
       placeholder="Buscar por nombre..." oninput="buscar(this.value)">
```

2. En app.js, una variable nueva en la parte de ESTADO y una línea más en cargarProductos:

```
let ultimaLista = [];          // en ESTADO: lo último que vino de la API

// en cargarProductos, después del res.json():
ultimaLista = productos;
```

3. La función que filtra:

```
function buscar(texto) {
  const t = texto.toLowerCase();
  const filtrados = ultimaLista.filter(
    p => p.nombre.toLowerCase().includes(t));
  pintarProductos(filtrados);
}
```

Código	Qué hace
`oninput`	llama a buscar cada vez que se escribe o se borra una letra
`toLowerCase()`	pasa todo a minúsculas: así “rayuela” encuentra “Rayuela”
`filter(...)`	se queda con los productos cuyo nombre incluye lo escrito

Como busca sobre ultimaLista, si elegiste una categoría el buscador busca solo dentro de ella.

Checklist de cierre de la clase

Antes de irte, verificá cada punto. Lo que no llegue hoy queda de tarea: la próxima clase arranca con el login.

Qué revisar	Cómo te das cuenta	Listo
Catálogo	los productos aparecen con imagen, categoría, precio y stock	☐
Sin stock	un producto con stock 0 muestra “Sin stock” y el botón apagado	☐
Filtro	cada categoría muestra solo sus productos y marca su botón	☐
F12 → Red	productos responde 200	☐
Carrito	suma, resta y sigue ahí después de recargar	☐
Tu rubro	nombre, colores y 6 productos propios con imagen	☐

✓ Recordá para el proyecto final

El catálogo con imágenes y filtro por categoría es parte del **mínimo obligatorio** del TP: lo que dejaste andando hoy ya cuenta para la entrega. La próxima clase conectamos el registro, el login y el botón Comprar, y la tienda queda completa de punta a punta.